Computer Shop Management System — Object-Oriented Programming Documentation

1. Overview

Java is an object-oriented language: programs are built from classes (blueprints) and objects (working instances of those blueprints). This system is designed around four pillars — Encapsulation, Inheritance, Polymorphism, and Abstraction — plus one architectural pattern, the DAO (Data Access Object) pattern, which applies those pillars at the scale of the whole program: all database code lives in dedicated classes, and the windows never write SQL directly.

The ten classes and their jobs:

- Windows: `Login`, `Dashboard`, `CustomerTimer` (all extend `JFrame`).
- Data access: `connectDB`, `LoginDAO`, `SessionDAO`, `PCsDAO`.
- Plain data plus logic: `Session` (POJO), `PasswordUtil` (hashing), `DataStructureManager` (structures).

2. Encapsulation

2.1 Definition

Bundling data together with the methods that operate on it, and hiding internal state behind access modifiers like `private`, so no outside code can corrupt it directly.

2.2 Where We Apply It

- `Session.java`: every field is `private`; all access goes through getters and setters.
- `DataStructureManager`: the `private Stack<ExtendRecord>` field can only be touched through `pushExtension` / `undoLastExtension`.
- DAOs: helpers like `mapRow()` and `logEvent()` are `private`, so callers cannot bypass the public API.
- Window internals (`pcIds`, `dsm`, timers) are `private` to their own frame; no other window can reach in and mutate them.

2.3 Code Example

One getter/setter pair from the live `Session.java`:

```
public int getCustomerId() {
    return customerId;
}
```

```
public void setCustomerId(int customerId) {
    this.customerId = customerId;
}
```

Outside code reads the value through `getCustomerId()` and never touches the field itself. If the internal representation ever changed, only these two methods would need updating. The `this.` prefix inside the setter resolves the name clash: `this.customerId` means "my field", bare `customerId` means "the incoming parameter" — the same name, two different variables, kept apart explicitly.

3. Inheritance

3.1 Definition

A subclass extends a superclass with the `extends` keyword and inherits all of its non-private members — fields and methods the subclass gets for free.

3.2 Where We Apply It

All three GUI classes extend `javax.swing.JFrame`:

```
public class Login extends javax.swing.JFrame {
public class Dashboard extends javax.swing.JFrame {
public class CustomerTimer extends javax.swing.JFrame {
```

Benefit: free window lifecycle methods. None of these classes implements `setVisible`, `dispose`, `pack`, or `setLocationRelativeTo` — every call to those methods in our code (for example `timer.setVisible(true)` in `Login`, or `this.dispose()` on logout) runs code inherited from `JFrame`. The subclass only adds what makes it unique: components, colors, event handlers, and database calls. A logout handler therefore reads as three inherited calls in a row — open the new window, center it, show it — plus one inherited close of the old one.

3.3 Code Example

The three declaration lines above, taken verbatim from `Login.java:3`, `Dashboard.java:4`, and `CustomerTimer.java:3`. Inheritance is visible in exactly one keyword per class — everything after it is reuse.

4. Polymorphism

4.1 Definition

Same name, different behavior. Two forms exist: overloading (same method name, different parameters — decided at compile time) and overriding (a subclass replaces an inherited method — decided at runtime). This codebase uses overloading.

4.2 Where We Apply It

Overloading: `CustomerTimer` has two constructors — a no-arg version and a `String`-arg version. The `String` version does everything the plain one does, plus it resolves the PC id from the username. Callers pick the behavior they need by choosing which signature to call.

Overriding (runtime replacement of an inherited method) is intentionally unused: no subclass here replaces a Swing method, so every inherited call behaves exactly as `JFrame` documents it.

At the call site, `new CustomerTimer()` opens a generic window while `new CustomerTimer(username)` opens one already bound to a PC — same class name, chosen behavior, no `if` statements required.

4.3 Code Example

Both constructors from the live `CustomerTimer.java` (lines 13–15 and 17–43):

```
public CustomerTimer() {
    this("customer");
}

public CustomerTimer(String username) {
    initComponents();
    setResizable(true);
    // ... window styling, sizing, centering ...
    if (username != null && !username.trim().isEmpty()) {
        userLabel.setText("Logged in as " + username.trim());
    }
    pack();
    setLocationRelativeTo(null);
    resolvePcId(username);
    tick();
    refreshTimer = new javax.swing.Timer(1000, e -> tick());
    refreshTimer.start();
}
```

`this("customer")` in the first constructor calls the second one with a dummy name — one constructor reusing the other instead of duplicating its body. `Login` always calls the `String` form with the real typed username. The compiler picks which constructor runs purely from the argument list: zero arguments means the first, one `String` means the second — that compile-time choice is what makes this overloading rather than overriding (nothing is inherited-and-replaced here).

5. Abstraction

5.1 Definition

Hiding complexity behind a simple interface: the caller states *what* it wants, never *how* it happens.

5.2 Where We Apply It

- `connectDB.getConnection()` hides driver loading (`Class.forName`), the JDBC URL format, and the username/password.
- The DAO layer hides SQL — GUI code calls `SessionDAO.startSession(...)` without knowing the INSERT statement, the placeholder binding, or the generated-keys handling inside.

5.3 Code Example

Before (what the GUI sees — one line in `Dashboard.doStart()`):

```
SessionDAO.startSession(pcId, hours, amount, customerId);
```

After (what that one line hides — inside `SessionDAO`):

```
String sql = "INSERT INTO sessions (pc_id, customer_id, hours, amount, start_time,
end_time, status, is_archived) VALUES (?, ?, ?, ?, NOW(), DATE_ADD(NOW(), INTERVAL ?
HOUR), 'active', 0)";
try (Connection conn = connectDB.getConnection();
     PreparedStatement pst = conn.prepareStatement(sql,
java.sql.Statement.RETURN_GENERATED_KEYS)) {
    pst.setInt(1, pcId);
    // ... bind remaining values, execute, read generated key, log START ...
}
```

Six GUI call sites across `Dashboard` and `CustomerTimer` share this pattern: one readable call outside, all JDBC mechanics inside. Other examples of the same hiding: `LoginDAO.validateLogin()` conceals the SELECT plus hash comparison, and `PCsDAO.getAllPCs()` conceals its query. Even `connectDB.getConnection()` is itself an abstraction one level down: callers never see driver loading, URL formatting, or credentials — they receive an open connection or `null`.

Second example, same pattern — login (`Login.SubmitRegActionPerformed`):

```
String role = LoginDAO.validateLogin(username, PasswordUtil.hashPassword(password));
```

hides:

```
String sql = "SELECT role FROM login_table WHERE username = ? AND password = ?";
// ... bind both values, return the role, or null when nothing matches ...
```

6. Class Diagram (UML)

[UML Class Diagram — image referenced in source]

The diagram above shows the class hierarchy and dependencies of the system. Solid lines represent inheritance, dashed arrows represent 'uses' dependencies. The Login window opens either Dashboard (for counter role) or CustomerTimer (for customer role). All data access flows through the DAO classes, which share a single connection helper.

7. DAO Pattern

7.1 What It Is

Data Access Object: a class whose only job is talking to one area of the database, so business logic (windows, timers, buttons) never contains SQL. The GUI asks *what*; the DAO decides *how*.

7.2 Where We Apply It

- `LoginDAO` → `login_table` (credential check only).
- `SessionDAO` → `sessions` + `session_log` (starts, extends, stops, undos, reads, totals, logs, customer lookup/registration).
- `PCsDAO` → `pcs` + `customers` (machine list, name lookup, broadcast flags).
- GUI classes never issue SQL directly — verified: after the Stage 3 refactor, zero `java.sql` imports remain in any window class.
- `connectDB` is used by all three DAOs and by nothing else; it owns the only credential strings in the program.
- `PCsDAO.getPendingBroadcast()` hides flag polling the same way; callers only ever see message-or-`null`.

7.3 Benefits

- Single responsibility per DAO: a sessions-table change touches `SessionDAO` and nothing else. During Stage 3, moving eight query sites out of `Dashboard` required zero changes to any button, label, or layout line.
- Easier to test: DAO methods take plain values and return plain values or POJOs, so each can be exercised without opening a window.
- Easier to swap databases without touching the GUI: only the DAO internals and `connectDB` know MySQL is underneath.

8. Summary Table

Concept	Where Demonstrated
Encapsulation	Session.java, DataStructureManager, DAOs
Inheritance	Login, Dashboard, CustomerTimer (all extend JFrame)
Polymorphism	CustomerTimer overloaded constructors
Abstraction	connectDB, DAO layer
DAO Pattern	LoginDAO, SessionDAO, PCsDAO

Submitted By:

RONALD ALLAN ZAMORA

BHONG S. KILATON

RYAN V. GESTA

KARL JOROSH C. PACULAN

ANSARY M. ASIS

AL KENNETH M. UNTALAN